BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Work growing logarithmically, as with repeated halving.

Q2. Extra memory beyond input storage.

Q3. No. It classifies asymptotic growth.

Q4. Six: 64→32→16→8→4→2→1.

You Do pages · Independent questions

Q1. O(1) [1], O(n) [1], O(n^2) [1], O(log n) [1].

Q2. Loop copies n elements [2], O(n) time [1]; new array size n [2], O(n) auxiliary space [1].

Q3. Bubble O(n^2) worst [2] and O(1) auxiliary in-place [1]; merge O(n log n) worst [2] but typically O(n) auxiliary storage [1].

Exit Ticket · Model answer

False. Nested loops may use only O(1) additional storage.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

RHS COMPUTING / OCR H446 / mixed LESSON 301 · DRAFT

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

LESSON 301 · Advanced Big O and space complexity

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What does O(log n) indicate?

Q2. What is auxiliary space?

Q3. Is Big O an exact millisecond prediction?

Q4. A sorted array of length64 is reduced by half after each unsuccessful search step. How many halvings reach length1?

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

asymptotic — Behaviour as input size grows.

Big O — Asymptotic upper bound.

auxiliary space — Storage additional to input.

polynomial — Growth such as n^2.

exponential — Growth such as 2^n.

New learning

Identify the dominant operation — Single full loop grows O(n); two full nested n loops grow $O(n^2)$. Repeated halving grows O(log n).

Separate time and space — Counting two scalar variables uses O(1) auxiliary memory even if repeated n^2 times; copying n elements uses O(n) extra storage.

Interpret asymptotics — Big O describes a bound on growth, not an exact runtime. Constants, cache effects and data size still matter.

Worked example

Find worst-case time and auxiliary space for summing n array elements into one accumulator.

1. Predict first — Each element is visited once: time O(n).

2. Explain the method — The only additional state is a scalar sum (apart from iterator control): auxiliary space O(1).

We Do · Practise with guidance

Find time and extra space for copying each element to a separate new n-element array.

```
int[] b=new int[a.Length];
for(int i=0;i<a.Length;i++) b[i]=a[i];
```

Predict first

Copy performs a constant amount of work n times, so time grows O(n).

Explain the method

New array b stores n values, giving O(n) auxiliary memory.

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Classify loops running once, n times, n^2 times and repeatedly halving n. [4]

Question 2

A function makes a new copy of every one of n elements. Identify time and auxiliary space. [6]

Question 3

Compare bubble sort and merge sort at large n using time and space trade-offs. [6]

Exit Ticket · Answer before checking

An $O(n^2)$ algorithm must always use $O(n^2)$ memory.